

LMJ AGENT · TECHNICAL REPORT

LMJ Agent

Telegram 기반 상시 AI 비서 · 자동화 시스템

개발 결과 보고 (기술팀) — 구성 · 운영 · 신뢰성

상시 가동 24/7

운영 스킬 9종

일 6회 자동 브리핑

launchd 잡 8종



2026-08-20 · macOS · Python · Bash · Claude Code



SUMMARY

한눈에 보기



24/7

상시 가동 에이전트

KeepAlive + 5분 헬스체크



9

운영 스킬

주식 4 · 날씨 3 · 항공 2



6

일일 자동 브리핑

주식 3회(평일) · 날씨 3회



8

launchd 잡

상시 2 + 정시 6



시스템 요약

- Telegram 채널로 24시간 대화·지시를 받는 상시 백그라운드 에이전트
- 주식·날씨 브리핑은 macOS launchd LaunchAgent가 정시에 헤드리스로 실행
- 기능은 스킬 9종으로 모듈화되어 단위별 추가·수정이 가능
- launchd · 런처 · heartbeat 3중 감시로 종료·행(hang) 시 자동 복구



운영 환경

실행 기반

macOS launchd + tmux 세션

채널

Telegram (MCP 플러그인)



추진 배경 및 목적



배경 (문제 정의)

- 시황·날씨·항공권 정보를 매번 직접 검색
- 정해진 시간대 반복 작업의 수작업 부담
- 모바일에서 즉시 받아볼 채널 부재
- 24시간 상시 응답하는 비서가 필요



목표

- Telegram으로 대화·지시 가능한 에이전트
- 정기 브리핑 완전 자동화 (아침/점심/저녁)
- 스킬 단위로 기능 확장 가능한 구조
- 장애 시 자동 복구되는 상시 운영



핵심 성과

- 상시 백그라운드 에이전트 구축
- 스킬 9종 운영 (주식·날씨·항공권)
- 일 6회 정기 브리핑 자동 발송
- 채널 기반 실시간 대화·요청 처리
- 자동 재시작·헬스체크 운영체계 확립



적용 현황

2026-06 구축 이후 단일 macOS 워크스테이션에서 상시 운영 중 · 정기 브리핑은 launchd LaunchAgent가 평일/매일 스케줄로 자동 발송



02 ARCHITECTURE

시스템 아키텍처 — 계층 구조



사용자 채널

Telegram · Discord — MCP 플러그인 채널 (인바운드 메시지 / 아웃바운드 발송)



에이전트 코어

Claude Code (auto-mode) · 스킬 라우팅 · 권한 allowlist · 메모리/파일 접근



스킬 & 도구

Skills 9종 · shared-scripts (market-fetcher · ticker-fetcher) · Playwright MCP · Web



데이터 · 출력

시장 / 날씨 / 항공 소스 → Telegram Bot API (실시간 · 정기 발송)



OS 기반

macOS launchd LaunchAgent 8종 (상시 2 + 정시 6) · tmux 상시 세션

계층 간 호출은 단방향(위 → 아래)이며, 결과 발송만 채널 계층으로 되돌아온다.



실행 모델 & 데이터 흐름



① 상시 대화형 에이전트

- launchd com.claude.agent — RunAtLoad + KeepAlive
- → agent-launcher.sh → tmux 세션(claude-agent)
- → claude --enable-auto-mode (Telegram 채널 연결)
- Telegram / Discord 메시지에 실시간 응답
- 종료·행(hang) 감지 시 자동 재시작



② 헤드리스 스케줄 실행

- launchd LaunchAgent 6종 — StartCalendarInterval
- → check-login.sh 프리플라이트 (로그인 점검)
- → claude -p [스킬] (1회성 헤드리스 실행)
- → tg-send.sh → Telegram Bot API 발송
- 인증 에러 감지 시 텔레그램 알림

트리거



스킬 실행



데이터 수집



LLM 가공



tg-send.sh



Bot API



사용자

수집 경로: market-fetcher.py(시장) · Playwright MCP(항공권 렌더링) · WebFetch/Search(폴백·뉴스)

프로세스: launchd 8종 + tmux | 설정: env.sh · settings.json · .mcp.json · *.yaml



03 SCHEDULER

launchd 잡 구성 (8종)

LABEL	트리거 키	주기	실행 대상
● <code>com.claude.agent</code>	<code>RunAtLoad + KeepAlive</code>	상시	<code>agent-launcher.sh</code> → <code>tmux</code> → <code>claude</code>
● <code>com.claude.agent.heartbeat</code>	<code>StartInterval 300</code>	5분	<code>heartbeat.sh</code> (hang 감지·복구)
● <code>com.ghyu.claude.stock.morning</code>	<code>StartCalendarInterval</code>	평일 07:30	<code>stock-cron.sh morning</code>
● <code>com.ghyu.claude.stock.lunch</code>	<code>StartCalendarInterval</code>	평일 12:30	<code>stock-cron.sh lunch</code>
● <code>com.ghyu.claude.stock.evening</code>	<code>StartCalendarInterval</code>	평일 17:30	<code>stock-cron.sh evening</code>
● <code>com.ghyu.claude.weather.morning</code>	<code>StartCalendarInterval</code>	매일 08:00	<code>weather-cron.sh morning</code>
● <code>com.ghyu.claude.weather.lunch</code>	<code>StartCalendarInterval</code>	매일 12:00	<code>weather-cron.sh lunch</code>
● <code>com.ghyu.claude.weather.evening</code>	<code>StartCalendarInterval</code>	매일 18:00	<code>weather-cron.sh evening</code>

수동 즉시 실행: `launchctl kickstart -k gui/$(id -u)/com.ghyu.claude.stock.lunch`

* 브리핑 스크립트 파일명은 `*-cron.sh` 이지만, 실행을 트리거하는 주체는 모두 `launchd`이다.



04 TECH STACK

기술 스택



런타임 / 에이전트

- Claude Code (auto-mode)
- Telegram · Discord MCP 채널
- Playwright MCP



언어 / 실행환경

- Python 3.14 (venv)
- Bash 스크립트
- Node / npx (MCP)



프로세스 관리

- launchd LaunchAgent 8종
- tmux 상시 세션
- heartbeat 헬스체크 (5분)



데이터 수집

- WebFetch / WebSearch
- Playwright (항공권)
- Yahoo Finance · Finviz



실행 환경

단일 macOS 워크스테이션에서 상시 가동 · Python 3.14 (venv) · Node 24 · tmux 세션 · launchd LaunchAgent 8종으로 모든 실행을 관리



05 RELIABILITY

에이전트 런타임 & 신뢰성

3중 감시 구조



1단계 launchd

com.claude.agent — KeepAlive:true / ThrottleInterval 10
프로세스가 죽으면 launchd가 즉시 재기동



2단계 agent-launcher.sh

30초 주기 헬스체크 · 세션/프로세스 종료 시 재시작
600초 윈도우 내 최대 10회 (재시작 폭주 방지)



3단계 heartbeat.sh

launchd가 5분마다 호출 — 살아있으나 응답 없는 hang 감지
2회 연속(10분) 감지 시 강제 종료 후 재시작



장애 대응 / 운영

- 로그 10MB 자동 로테이션 (agent.log)
- 로그인 실패 시 5분 주기 재시도 + 텔레그램 알림 (쿨다운 30분)
- STARTUP_GRACE 30초 — 인증·크리덴셜 로드 대기 후 헬스체크 시작
- 스케줄 잡은 check-login.sh 프리플라이트로 사전 차단 (비용 절감)
- 실행 결과에서 인증 에러 문자열 감지 → 서버측 토큰 무효까지 포착
- logs/.hang_count 마커로 연속 hang 횟수를 추적·리셋
- 잡별 로그 분리 — agent · heartbeat · stock · weather
- 운영 스크립트: agent-start.sh / agent-stop.sh / agent-attach.sh



스킬 시스템 (9종)



주식 (4)

STOCK

- stock-morning / lunch / evening
- → 장 시간대별 시황 정기 브리핑
- stock-analyze → 단타·스윙 진입 분석
- market-fetcher · ticker-fetcher 공통 사용



날씨 (3)

WEATHER

- weather-morning / lunch / evening
- → 시간대별 날씨 브리핑
- 저녁 브리핑에 내일 날씨 포함



항공권 (2)

FLIGHT

- flight-check → 최저가 실시간 조회
- flight-watch → 매진편 좌석 감시·알림
- Playwright MCP로 네이버 항공권 렌더링



공통 구조

SKILL.md(실행 지침) + shared-scripts(수집 로직) 조합. 스킬 단위로 독립 추가·수정이 가능해 기능 확장 비용이 낮다.



07 PIPELINE

주식 브리핑 파이프라인

launchd 정시 트리거 → 스킵 실행 → 데이터 수집 → 메시지 작성 → Telegram 전송

1 트리거

- launchd LaunchAgent
- 평일 07:30 / 12:30 / 17:30
- check-login.sh 프리플라이트
- stock-cron.sh 호출

2 수집

- market-fetcher.py
- 지수 · 선물 · 환율 · 섹터
- 배치 API로 일괄 수집
- 실패 시 Web 폴백

3 가공

- 주도 테마 TOP3
- 매수 / 매도 종목 선정
- 실명 + 종목코드 표기
- 다음 세션 관전 포인트

4 전송

- 모바일 가독성 포맷
- 표 대신 인라인 구성
- 2,000자 이내로 압축
- tg-send.sh → Bot API



날씨 파이프라인

동일한 4단계 구조를 그대로 사용하며, 트리거만 weather LaunchAgent 3종(매일 08:00 / 12:00 / 18:00)으로 바뀐다.



07-1 SCREENSHOT

Telegram 브리핑 — 주식 · 날씨

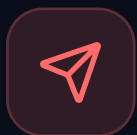


▲ 주식 모닝 브리핑 (평일 07:30)



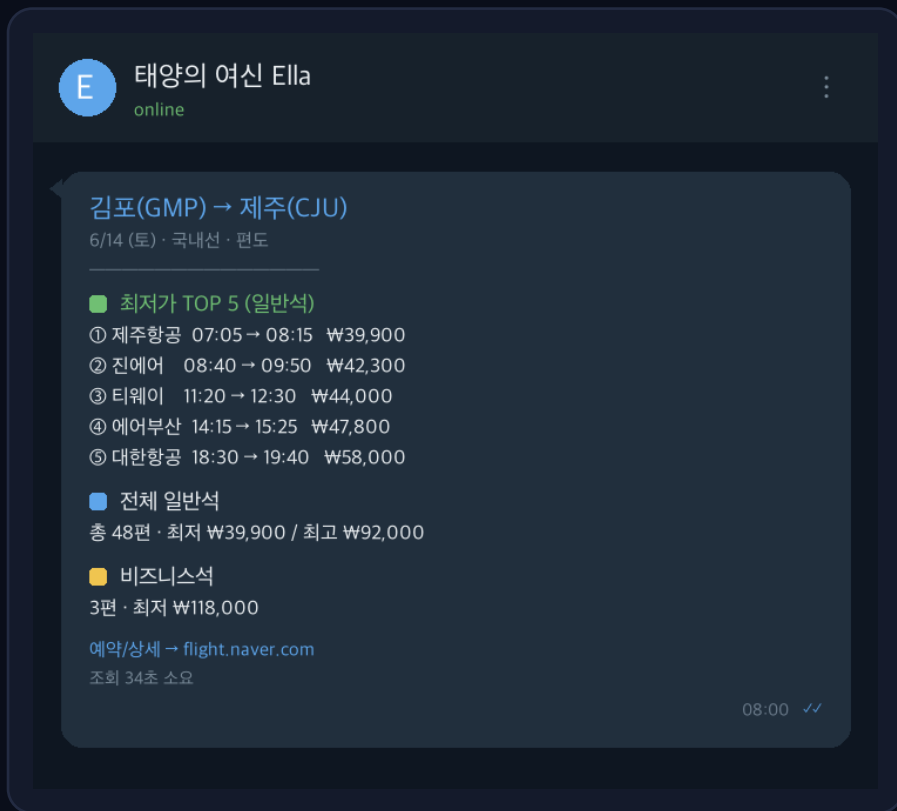
▲ 날씨 모닝 브리핑 (매일 08:00)

* 실제 발송 데이터를 기반으로 재구성한 화면

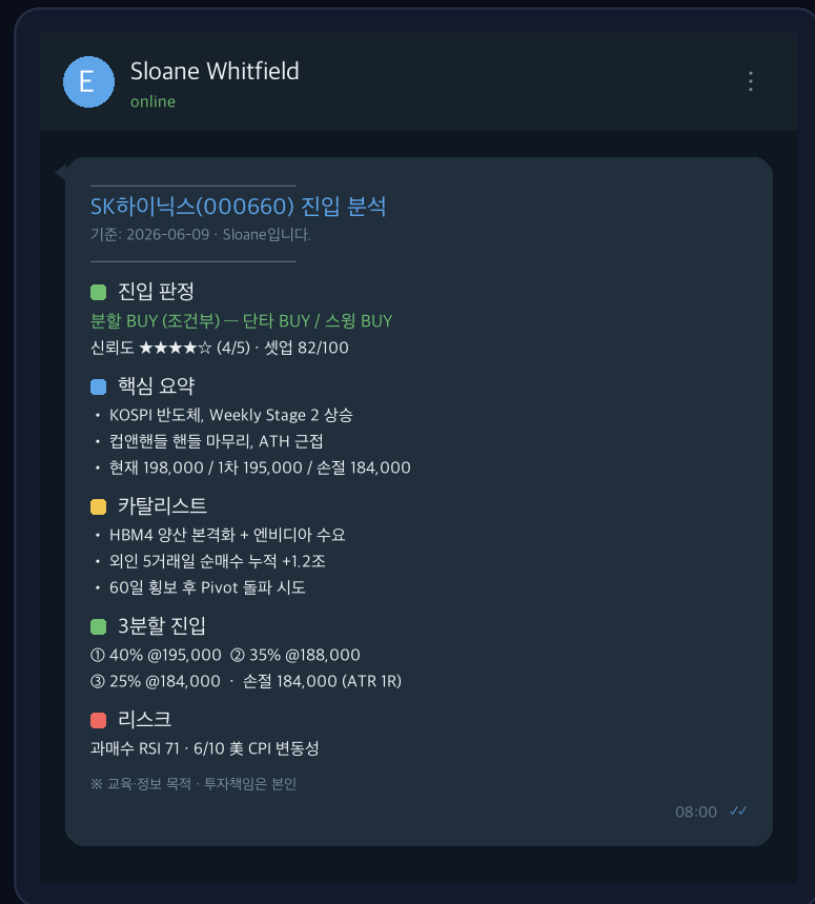


07-2 SCREENSHOT

Telegram — 항공권 · 종목 분석



▲ 항공권 최저가 조회 (flight-check)



▲ 종목 진입 분석 (stock-analyze)

* 실제 발송 포맷을 기반으로 재구성한 화면



자동화 & 구성 관리



스케줄 작업

- 주식 브리핑 — 평일 07:30 / 12:30 / 17:30
- 날씨 브리핑 — 매일 08:00 / 12:00 / 18:00
- heartbeat 헬스체크 — 5분 주기
- 에이전트 상시 구동 · 자동 복구



실행 메커니즘

- launchd — 상시 데몬 + 정시 트리거
- check-login.sh — 실행 전 인증 점검
- tg-send.sh — 공통 Telegram 전송 유틸
- /loop — 감시형 작업 반복 실행



구성 관리

- .claude/settings.json — 권한 allowlist
- config/env.sh — 공통 환경변수
- .mcp.json — Playwright MCP 등록
- 스킬별 SKILL.md — 실행 지침
- LaunchAgents/*.plist — 잡 정의



단일 운영 체계

정시 실행 · 상시 구동 · 헬스체크가 모두 launchd 한 체계에서 관리되며, 잡 정의는 plist 파일로 버전 관리·재현이 가능하다.



보안 · 운영 고려사항



현재 적용

- 채널 접근제어 — access 스킴 기반 페어링 승인
- 도구 권한 allowlist (.claude/settings.json)
- 파일 접근 범위를 프로젝트 디렉토리로 제한
- 파괴적 작업(rm · 덮어쓰기) 전 확인 정책
- launchd plist에서 환경변수·작업디렉토리 고정



개선 과제

- API 키 평문 저장 → 환경변수 / 시크릿 분리 필요
- auto-mode · skip-permissions 적용 범위 재점검
- 로그 내 민감정보 마스킹 규칙 부재
- 외부 전송 데이터 검토 프로세스 미비
- 단일 계정 keychain 의존 — 만료 시 전체 스케줄 중단



조치 우선순위

- ① 시크릿(.env / keychain) 분리 ② 도구 권한 범위 축소 ③ 로그 마스킹 규칙 수립 — keychain 만료 알림이 최우선



성과 · 한계 · 향후 계획



성과

- 정기 브리핑 수작업 → 완전 자동화
- 모바일에서 즉시 정보 수신
- 9개 스킬로 기능 모듈화
- 3중 감시로 자동복구 상시 운영 달성
- launchd 단일 체계로 스케줄 운영



한계

- 단일 macOS 머신 의존 (SPOF)
- 외부 사이트 구조 변경에 취약
- 보안 강화 필요 (09장 참조)
- 테스트 · 모니터링 자동화 부족



향후 계획

- 시크릿 관리 체계화
- 스킬 추가 (캘린더 · 메일 등)
- 장애 알림 · 운영 지표 모니터링
- 배포 / 이중화 검토



다음 마일스톤

시크릿 분리와 운영 지표 모니터링을 우선 적용하고, 안정화가 확인되면 이중화(보조 실행 환경)를 검토한다.

THANK YOU

감사합니다

Q & A · 실 서비스 데모 시연 가능

